

NumPy, mapped to your ML syllabus

Every function here earns its place in one of your five units — Preprocessing, Regression, Classification, or Clustering — with a definition, every practical use case, worked examples, and an exercise. No filler functions you won't touch this semester.

0.12

0.98

1.00

0.44

0.07

0.61

`array([0.12, 0.98, 1.00, 0.44, 0.07, 0.61])` — a NumPy array is the data structure every model in this course is built on.

BEFORE UNIT 1

Foundations you need for every unit

These aren't in one specific unit — they're the grammar the rest of the syllabus is written in. Master these four before touching regression or clustering code.

`np.array()` `np.array(object, dtype=None)`

DEFINITION

Creates a NumPy `ndarray` (n-dimensional array) from a Python list, tuple, or nested sequence. This is the single data structure your feature matrix X , target vector y , and weight vector θ will be stored in throughout the course.

USES IN THIS SYLLABUS

- Wrapping a dataset (rows = samples, columns = features) as a 2-D array for regression/classification.
- Building small vectors for distance calculations in KNN and clustering.
- Converting a Pandas column to an array before feeding it into a formula.

EVERY CASE

1. 1-D array (a single feature or a target vector y)

```
y = np.array([3.1, 4.2, 5.0, 2.8])
# y.shape -> (4,)
```

2. 2-D array (a feature matrix X — rows=samples, cols=features)

```
X = np.array([[1500, 3], [1600, 3], [1700, 4]])
# X.shape -> (3, 2) → 3 houses, 2 features (area, rooms)
```

3. Explicit dtype (important before matrix math — avoid integer division bugs)

```
theta = np.array([0, 0], dtype=float)
```

0	1	2	3
3.1	4.2	5.0	2.8

A 1-D array y — index 2 (highlighted) holds the value 5.0.

np.zeros() • np.ones() • np.eye()

```
np.zeros(shape) / np.ones(shape) / np.eye(N)
```

DEFINITION

zeros/ones create arrays filled with 0s or 1s of a given shape; eye(N) creates an N×N identity matrix (1s on the diagonal, 0s elsewhere).

USES IN THIS SYLLABUS

- Initializing regression weights θ to zero before gradient descent.
- Adding a "bias column" of ones to X for the intercept term in linear/logistic regression.
- The identity matrix appears in the regularization (ridge regression) formula and in PCA-related linear algebra.

1. Zero-initialize weights for gradient descent

```
theta = np.zeros(3)    # [0. 0. 0.] – one weight per feature + bias
```

2. Add an intercept (bias) column of ones to X

```
ones_col = np.ones((X.shape[0], 1))  
X_bias = np.hstack([ones_col, X])
```

3. Identity matrix for Ridge regression's closed-form solution

```
lam = 0.5  
I = np.eye(X.shape[1])  
theta = np.linalg.inv(X.T @ X + lam * I) @ X.T @ y
```

`np.arange()` • `np.linspace()`

```
np.arange(start, stop, step) / np.linspace(start, stop, num)
```

DEFINITION

`arange` generates values in a range using a step size (like Python's `range`, but returns an array). `linspace` generates a fixed *number* of evenly spaced values between two endpoints.

USES IN THIS SYLLABUS

- Generating a smooth range of x-values to plot a fitted regression line.
- Creating a grid of candidate values to try for hyperparameters (e.g. k in KNN, λ in ridge/lasso).
- Building synthetic toy datasets to test an algorithm before using a real one from data.gov.in / UCI.

1. Sweep of k values to test in K-NN

```
k_values = np.arange(1, 21, 2)    # array([1,3,5,...,19]) – odd k avoids ties
```

2. Smooth x-axis to draw a fitted regression curve

```
x_line = np.linspace(X.min(), X.max(), 100)
```

`.shape` / `.reshape()` / `.ndim` `array.reshape(new_shape)`

DEFINITION

`.shape` reports an array's dimensions as a tuple; `.reshape()` returns the same data viewed with a new shape (no data is copied unless necessary); `.ndim` reports the number of dimensions.

USES IN THIS SYLLABUS

- Turning a flat 1-D feature array into a column vector so it matches the (samples, features) shape scikit-style formulas expect.
- Reshaping a 1-D distance array before sorting for KNN.
- Flattening a 2-D image/feature block into a 1-D vector for a perceptron/neural network input layer.

1. Turn a single feature column into 2-D shape (n, 1)

```
area = np.array([1500, 1600, 1700])
area = area.reshape(-1, 1)
# shape becomes (3, 1) - "-1" means "figure this dimension out"
```

2. Flatten a 2-D array to feed into a perceptron

```
patch = np.array([[1, 0], [0, 1]])
flat = patch.reshape(-1) # array([1, 0, 0, 1])
```

Indexing & Slicing `a[i]`, `a[i:j]`, `a[:, k]`, `a[mask]`

DEFINITION

NumPy supports positional indexing (`a[2]`), slicing ranges (`a[1:4]`), column/row selection in 2-D arrays (`a[:, 0]`), and **boolean masking** — selecting elements that satisfy a condition (`a[a > 5]`).

USES IN THIS SYLLABUS

- Splitting a dataset into a train/test set by row slicing.
- Pulling out a single feature column for feature scaling.
- Boolean masking is exactly how you compute confusion-matrix quantities (TP/FP/TN/FN) for classification evaluation.

1. Train/test split by row slicing

```
split = int(0.8 * len(X))
X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]
```

2. Select one feature column (2-D array)

```
area_col = X[:, 0]  # every row, column 0
```

3. Boolean masking to count classification outcomes

```
TP = np.sum((y_pred == 1) & (y_true == 1))
FP = np.sum((y_pred == 1) & (y_true == 0))
```

0	1	2	3	4
12	45	38	9	51

`a[a > 20]` → selects the highlighted cells → `array([45, 38, 51])`

UNIT 2 • 6 HOURS

Preprocessing — Feature Scaling & PCA

Before any model sees your data, it needs to be scaled and, often, compressed into fewer dimensions. These functions cover feature scaling (Ch.6) and Principal Component Analysis (Ch.12.2).

`np.mean()` • `np.std()` • `np.var()`

```
np.mean(a, axis=None) / np.std(a, axis=None)
```

DEFINITION

`mean` computes the average, `var` the variance (average squared deviation from the mean), and `std` the standard deviation (square root of variance). The `axis` parameter controls direction: `axis=0` collapses rows (one result per column/feature), `axis=1` collapses columns (one result per row/sample).

USES IN THIS SYLLABUS

- **Standardization / Z-score scaling:** $(x - \text{mean}) / \text{std}$ — the core feature-scaling formula in Ch.6.1.1.
- **PCA:** PCA requires mean-centering the data before computing the covariance matrix.
- Reporting average error (MSE) in regression evaluation.

1. Standardize every feature column (axis=0 → per-column stats)

```
mu = X.mean(axis=0)
sigma = X.std(axis=0)
X_scaled = (X - mu) / sigma
# each column now has mean≈0, std≈1
```

2. Mean-center data before PCA

```
X_centered = X - X.mean(axis=0)
```

3. Mean Squared Error for regression evaluation

```
mse = np.mean((y_true - y_pred) ** 2)
```

Watch axis carefully — axis=0 gives per-feature stats (what scaling needs); axis=1 gives per-sample stats. Getting this backwards is the #1 preprocessing bug.

np.min() • np.max() • np.ptp()

```
a.min(axis=0) / a.max(axis=0) / a.ptp(axis=0)
```

DEFINITION

min/max return the smallest/largest values along an axis; ptp ("peak to peak") returns max - min in one call.

USES IN THIS SYLLABUS

- **Min-Max scaling** (normalization): rescale every feature into $[0, 1]$, an alternative to standardization mentioned in Ch.6.1.2.
- Setting axis limits when plotting decision boundaries or regression lines.

1. Min-Max normalize each feature to $[0, 1]$

```
X_norm = (X - X.min(axis=0)) / (X.max(axis=0) - X.min(axis=0))
```

2. Same idea using ptp()

```
X_norm = (X - X.min(axis=0)) / X.ptp(axis=0)
```

np.cov()

```
np.cov(m, rowvar=True)
```

DEFINITION

Computes the covariance matrix: entry (i, j) measures how features i and j vary together. By default NumPy treats each *row* as a variable — for a standard (samples $\times$ features) matrix, set `rowvar=False`.

USES IN THIS SYLLABUS

- The covariance matrix is the direct input to PCA (Ch.12.2) — its eigenvectors are the principal components.

1. Covariance matrix of a (samples $\times$ features) dataset

```
cov_matrix = np.cov(X_centered, rowvar=False)
# shape: (n_features, n_features)
```

np.linalg.eig()

```
eigvals, eigvecs = np.linalg.eig(A)
```

DEFINITION

Computes the eigenvalues and eigenvectors of a square matrix A . Each eigenvector points in a direction the matrix only stretches (doesn't rotate); the matching eigenvalue is the stretch factor.

USES IN THIS SYLLABUS

- This is the heart of PCA:** the eigenvectors of the covariance matrix are the principal components (new axes); the eigenvalues tell you how much variance each component explains.

1. Full PCA pipeline using mean, cov, and eig

```
X_centered = X - X.mean(axis=0)
cov_matrix = np.cov(X_centered, rowvar=False)
eigvals, eigvecs = np.linalg.eig(cov_matrix)
```

```
# sort components by explained variance, descending
order = np.argsort(eigvals)[::-1]
eigvals, eigvecs = eigvals[order], eigvecs[:, order]

# keep top 2 principal components
top2 = eigvecs[:, :2]
X_pca = X_centered @ top2
```

2. Percentage of variance explained by each component

```
explained_ratio = eigvals / eigvals.sum()
```

np.linalg.svd() `U, S, Vt = np.linalg.svd(A)`

DEFINITION

Singular Value Decomposition factors any matrix into $U \cdot S \cdot V^T$. It's a numerically more stable alternative route to the same principal directions PCA needs, without ever explicitly forming the covariance matrix.

USES IN THIS SYLLABUS

- Alternative implementation of PCA for dimensionality reduction (Ch.12.2) — useful to know both routes exist.

1. PCA via SVD instead of eig

```
X_centered = X - X.mean(axis=0)
U, S, Vt = np.linalg.svd(X_centered, full_matrices=False)
X_pca = X_centered @ Vt[:2].T # project onto top 2 components
```

UNIT 2 EXERCISE — PREPROCESSING

1. Load any numeric dataset (e.g. from archive.ics.uci.edu/ml/datasets.php) into an array X of shape (samples, features).
2. Standardize X using mean/std, then separately Min-Max scale it. Print both and compare their ranges.
3. Mean-center X, compute its covariance matrix with `np.cov`, then find its eigenvalues/eigenvectors with `np.linalg.eig`.

4. Project the data onto the top 2 principal components and report what percentage of total variance they explain.

UNIT 3 • 12 HOURS

Regression — Linear Regression & Gradient Descent

Simple and multiple linear regression, gradient descent, over-fitting/regularization, and evaluation metrics (Ch.3.1–3.2, Ch.6.2.1) all reduce to matrix arithmetic. These five functions cover essentially all of it.

`np.dot()` • `np.matmul()` • `@` `np.dot(a, b) ≡ a @ b`

DEFINITION

Computes the matrix/vector dot product. For two 1-D vectors it's the scalar dot product; for 2-D arrays it's standard matrix multiplication. The `@` operator is shorthand for the same operation and is the version you'll use most.

USES IN THIS SYLLABUS

- **Hypothesis function** of linear regression: $\hat{y} = X @ \text{theta}$ computes predictions for every sample at once — no Python loop needed.
- Computing the normal equation solution for regression weights.
- Projecting data onto principal components (Unit 2) and computing similarity/distance scores.

1. Multiple linear regression prediction (vectorized — no for-loop)

```
theta = np.array([2.0, 0.05, 1.2]) # bias, weight1, weight2
X_bias = np.hstack([np.ones((X.shape[0],1)), X])
y_pred = X_bias @ theta
```

2. Cost function (MSE) using dot product for the sum of squares

```
errors = y_pred - y_true
cost = (errors @ errors) / (2 * len(y_true))
```

`np.transpose()` • `.T` `a.T` $\equiv$ `np.transpose(a)`

DEFINITION

Flips an array's axes — rows become columns and vice versa. For a 2-D matrix, (m, n) becomes (n, m) .

USES IN THIS SYLLABUS

- Required in the **Normal Equation**: $\theta = (X^T X)^{-1} X^T y$.
- Computing gradients in gradient descent, where you need $X^T \cdot \text{error}$.

1. Normal equation for linear regression (closed-form solution)

```
theta = np.linalg.inv(X_bias.T @ X_bias) @ X_bias.T @ y
```

2. Gradient of the MSE cost function

```
m = len(y)
gradient = (1/m) * (X_bias.T @ (X_bias @ theta - y))
```

`np.linalg.inv()` `np.linalg.inv(A)`

DEFINITION

Computes the matrix inverse A^{-1} such that $A @ A^{-1} = I$. Only works on square, non-singular matrices.

USES IN THIS SYLLABUS

- Solving linear regression directly via the Normal Equation, without gradient descent.
- The closed-form solution for Ridge Regression (L2 regularization), where the identity matrix (`np.eye`) is added before inverting to fix over-fitting.

1. Plain normal equation

```
theta = np.linalg.inv(X_bias.T @ X_bias) @ X_bias.T @ y
```

2. Ridge regression (regularized) closed-form solution

```
lam = 1.0
I = np.eye(X_bias.shape[1])
theta_ridge = np.linalg.inv(X_bias.T @ X_bias + lam * I) @ X_bias.T @ y
```

If `np.linalg.inv` raises `LinAlgError: Singular matrix`, your features are likely perfectly correlated (multicollinearity) – a real symptom of over-fitting risk covered in Ch.6.2.1.

`np.sum()` `np.sum(a, axis=None)`

DEFINITION

Adds up array elements, optionally along a given axis.

USES IN THIS SYLLABUS

- Computing the cost function (sum of squared errors) in linear regression.
- Computing regression evaluation metrics: MSE and R^2 .
- Summing gradients across all training samples in gradient descent.

1. R^2 score (coefficient of determination) – regression evaluation metric

```
ss_res = np.sum((y_true - y_pred) ** 2)
ss_tot = np.sum((y_true - y_true.mean()) ** 2)
r2 = 1 - (ss_res / ss_tot)
```

2. Batch gradient descent loop

```
m, lr, epochs = len(y), 0.01, 1000
theta = np.zeros(X_bias.shape[1])
for _ in range(epochs):
    error = X_bias @ theta - y
    grad = (1/m) * (X_bias.T @ error)
    theta -= lr * grad
```

`np.random.randn()` • **`np.random.rand()`**

`np.random.randn(*shape) / np.random.rand(*shape)`

DEFINITION

`randn` draws samples from the standard normal distribution (mean 0, std 1); `rand` draws uniformly from $[0, 1)$.

USES IN THIS SYLLABUS

- Randomly initializing weights for gradient descent, perceptrons, and neural networks (small random values avoid symmetry problems that zero-init causes in multi-layer networks).
- Shuffling / generating synthetic datasets for testing before touching real data.
- Random initial centroids for K-means (Unit 5).

1. Random weight initialization for a neural network layer

```
np.random.seed(42)    # reproducibility
weights = np.random.randn(4, 3) * 0.01    # small random values
```

2. Shuffle a dataset before splitting into train/test

```
indices = np.random.permutation(len(X))
X_shuffled, y_shuffled = X[indices], y[indices]
```

UNIT 3 EXERCISE — REGRESSION

1. Implement simple linear regression (1 feature) using the Normal Equation with `np.linalg.inv`, `.T`, and `@`.
2. Implement the same problem with batch gradient descent (a Python loop + `np.sum/@`) and confirm both give nearly the same `theta`.
3. Extend to multiple linear regression with 3+ features from a UCI/data.gov.in dataset.
4. Add L2 regularization (Ridge) using `np.eye`, and compare the R^2 and MSE (using `np.sum/np.mean`) against the unregularized model on a held-out test split.

UNIT 4 • 15 HOURS

Classification — From Logistic Regression to SVM

Decision trees, Naive Bayes, logistic regression, KNN, perceptron/MLP/neural networks, and SVM (Ch.3, 4.3, 5.1, 8, 9, 10, 19.7) share a small toolkit: exponentials, logs, distances, and `argmax/argmin`.

np.exp()

np.exp(x)

DEFINITION

Computes e raised to the power of each element — the exponential function, applied element-wise.

USES IN THIS SYLLABUS

- **Sigmoid activation** in logistic regression (Ch.4.3) and the perceptron/neural network activation functions (Ch.9).
- The Softmax function for multi-class outputs in neural networks.
- The Gaussian probability density used inside Naive Bayes for continuous features (Ch.8).

1. Sigmoid function for logistic regression

```
def sigmoid(z):  
    return 1 / (1 + np.exp(-z))  
  
z = X_bias @ theta  
probabilities = sigmoid(z)
```

2. Gaussian likelihood for Naive Bayes with continuous features

```
def gaussian_prob(x, mean, var):  
    coeff = 1 / np.sqrt(2 * np.pi * var)  
    exponent = np.exp(-((x - mean) ** 2) / (2 * var))  
    return coeff * exponent
```

3. Softmax for multi-class neural network output layer

```
def softmax(z):  
    exp_z = np.exp(z - np.max(z))    # subtract max for numerical stability  
    return exp_z / np.sum(exp_z)
```

np.log()

np.log(x)

DEFINITION

Computes the natural logarithm element-wise.

USES IN THIS SYLLABUS

- **Log-loss / cross-entropy** — the cost function that trains logistic regression and neural network classifiers.
- Log-probabilities in Naive Bayes, which avoid numerical underflow when multiplying many small probabilities together.

1. Binary cross-entropy (log-loss) cost function for logistic regression

```
m = len(y)
cost = -(1/m) * np.sum(
    y * np.log(probabilities) + (1 - y) * np.log(1 - probabilities)
)
```

2. Naive Bayes using log-probabilities (avoids underflow)

```
log_posterior = np.log(prior) + np.sum(np.log(likelihoods))
```

np.where() `np.where(condition, value_if_true, value_if_false)`

DEFINITION

Element-wise conditional selection — returns one value where a condition holds and another where it doesn't.

USES IN THIS SYLLABUS

- Converting logistic regression / SVM probability or score outputs into hard class labels (0/1) using a decision threshold.
- Implementing the perceptron's step activation function.

1. Threshold probabilities into class labels

```
y_pred_labels = np.where(probabilities >= 0.5, 1, 0)
```

2. Perceptron step activation

```
def step_activation(z):
    return np.where(z >= 0, 1, 0)
```

3. SVM sign decision function

```
y_pred = np.where(X_bias @ theta >= 0, 1, -1)
```

`np.argmax()` • `np.argmin()`

```
np.argmax(a, axis=None) / np.argmin(a, axis=None)
```

DEFINITION

Return the *index* of the maximum (or minimum) value along an axis — not the value itself.

USES IN THIS SYLLABUS

- Choosing the winning class in multi-class classification (which class has the highest predicted probability).
- Finding the split feature with the best information gain / lowest Gini impurity in a Decision Tree (Ch.10).
- Finding the nearest neighbor's index in KNN.

1. Multi-class prediction from a softmax output

```
probs = np.array([0.1, 0.7, 0.2])  
predicted_class = np.argmax(probs)    # 1
```

2. Decision tree split selection — pick the feature with lowest impurity

```
best_feature = np.argmin(gini_scores)
```

`np.linalg.norm()` `np.linalg.norm(x, ord=None, axis=None)`

DEFINITION

Computes vector/matrix norms. With default settings on a vector, it computes the Euclidean (L2) distance/length. Set `ord=1` for Manhattan distance.

USES IN THIS SYLLABUS

- **K-Nearest Neighbor classifier** (Ch.9): computing the distance from a query point to every training point is the entire algorithm.
- Also reused identically in Unit 5 for K-means / hierarchical clustering distance metrics.

1. Euclidean distance from one query point to all training points (vectorized)

```
query = np.array([5.1, 3.5])  
distances = np.linalg.norm(X_train - query, axis=1)
```

2. Manhattan distance (ord=1) — an alternative KNN distance metric

```
distances_manhattan = np.linalg.norm(X_train - query, ord=1, axis=1)
```

np.sort() • np.argsort()

np.sort(a) / np.argsort(a)

DEFINITION

`sort` returns the array's values in ascending order; `argsort` returns the *indices* that would sort it — far more useful when you need to know which original row was closest.

USES IN THIS SYLLABUS

- K-Nearest Neighbor: after computing all distances, sort them to find the k closest training points.
- Ranking principal components by explained variance in PCA (Unit 2).

1. Full KNN prediction using `linalg.norm` + `argsort` + majority vote

```
k = 5
distances = np.linalg.norm(X_train - query, axis=1)
nearest_idx = np.argsort(distances)[:k]
nearest_labels = y_train[nearest_idx]
```

np.unique() • np.bincount()

np.unique(a, return_counts=True) / np.bincount(a)

DEFINITION

`unique` returns the sorted distinct values in an array (optionally with their counts); `bincount` counts occurrences of each non-negative integer, which is the fastest way to do majority voting.

USES IN THIS SYLLABUS

- Finding class labels and their prior probabilities for Naive Bayes.
- Majority-vote step of the K-Nearest Neighbor classifier.

1. Class priors for Naive Bayes


```
classes, counts = np.unique(y_train, return_counts=True)
priors = counts / counts.sum()
```

2. Majority vote to finish the KNN prediction from the previous card

```
predicted_class = np.argmax(np.bincount(nearest_labels))
```

np.maximum() `np.maximum(x1, x2)`

DEFINITION

Element-wise maximum of two arrays (or an array and a scalar) — different from `np.max`, which reduces one array to a single value.

USES IN THIS SYLLABUS

- **ReLU activation**, the standard hidden-layer activation for multilayer perceptrons / neural networks (Ch.9).
- Hinge loss computation for SVM (Ch.19.7).

1. ReLU activation for a neural network hidden layer

```
def relu(z):
    return np.maximum(0, z)
```

2. Hinge loss for SVM

```
margins = np.maximum(0, 1 - y_true * y_pred_raw)
hinge_loss = np.mean(margins)
```

UNIT 4 EXERCISE – CLASSIFICATION

1. Implement logistic regression from scratch: sigmoid (`np.exp`), log-loss (`np.log`, `np.sum`), gradient descent, and threshold predictions with `np.where`.
2. Implement a K-Nearest Neighbor classifier using only `np.linalg.norm`, `np.argsort`, and `np.bincount` — no library KNN allowed.
3. Implement a single perceptron with a step activation (`np.where`) that learns the AND/OR logic gate.

4. Compute Accuracy, Precision, Recall, Specificity, F1-score for one of your classifiers using boolean-mask counting (see Foundations section).

UNIT 5 • 7 HOURS

Clustering — K-Means & Hierarchical Clustering

Unsupervised learning drops the labels but keeps the same distance math from Unit 4. K-means adds one new idea: iteratively recomputing centroids (Ch.12.4.1, 12.4.2).

`np.mean()` – for centroid updates `cluster_points.mean(axis=0)`

DEFINITION

Same function as in Unit 2 — the average, here applied per-cluster to relocate each centroid to the middle of its assigned points.

USES IN THIS SYLLABUS

- **K-means "update" step:** after assigning every point to its nearest centroid, the new centroid is the mean of all points in that cluster.

1. Recompute each of k centroids

```
new_centroids = np.array([
    X[labels == c].mean(axis=0) for c in range(k)
])
```

`np.linalg.norm()` + `np.argmin()` – for cluster assignment

```
np.argmin(distances, axis=1)
```

DEFINITION

The same distance (`linalg.norm`) and index-of-minimum (`argmin`) functions from Unit 4, now used to assign each point to its *closest centroid* rather than its closest neighbor.

USES IN THIS SYLLABUS

- **K-means "assignment" step** and computing the distance metrics that hierarchical clustering's linkage criteria are built on (Ch.12.4).

1. Assign every point to its nearest of k centroids (fully vectorized)

```
# distances shape: (n_samples, k)
distances = np.linalg.norm(X[:, None, :] - centroids[None, :, :], axis=2)
labels = np.argmin(distances, axis=1)
```

2. Complete K-means loop (assignment + update, from scratch)

```
for iteration in range(max_iters):
    distances = np.linalg.norm(X[:, None, :] - centroids[None, :, :], axis=2)
    labels = np.argmin(distances, axis=1)
    new_centroids = np.array([X[labels == c].mean(axis=0) for c in range(k)])
    if np.allclose(new_centroids, centroids):
        break
    centroids = new_centroids
```

np.random.choice()

np.random.choice(a, size, replace=False)

DEFINITION

Randomly samples elements (or indices) from an array, with or without replacement.

USES IN THIS SYLLABUS

- Randomly picking k initial centroids from the dataset to start K-means (Ch.12.4.1) — the standard initialization approach.

1. Random centroid initialization for K-means

```
k = 3
initial_idx = np.random.choice(len(X), size=k, replace=False)
centroids = X[initial_idx]
```

UNIT 5 EXERCISE — CLUSTERING

1. Implement K-means fully from scratch using only `np.random.choice`, `np.linalg.norm`, `np.argmin`, and `np.mean`.

2. Run it with $k = 2, 3$, and 4 on the same dataset. Plot (or print) how the cluster assignments change.
3. Compute the within-cluster sum of squared distances for each k to informally observe the "elbow" that motivates choosing a good k .
4. Compare the pairwise distance matrix produced by Euclidean (`ord=2`, default) vs. Manhattan (`ord=1`) norms — this pairwise matrix is exactly what agglomerative hierarchical clustering builds its linkages from.

Full Cheat-Sheet

Every function in these notes, at a glance — grouped by unit for quick pre-exam review.

FUNCTION	UNIT	ONE-LINE PURPOSE
<code>np.array</code>	Foundations	Create an ndarray from a list/nested list
<code>np.zeros/ones/eye</code>	Foundations	Init weights, bias column, identity matrix
<code>np.arange/linspace</code>	Foundations	Ranges of values (hyperparameter sweeps, plotting)
<code>.reshape/.shape</code>	Foundations	Change/inspect array dimensions
Indexing/slicing/masking	Foundations	Train/test split, feature selection, confusion counts
<code>np.mean/std/var</code>	Unit 2	Standardization, PCA mean-centering, MSE
<code>np.min/max/ptp</code>	Unit 2	Min-max normalization
<code>np.cov</code>	Unit 2	Covariance matrix for PCA
<code>np.linalg.eig</code>	Unit 2	Principal components (eigenvectors) + variance (eigenvalues)
<code>np.linalg.svd</code>	Unit 2	Stable alternative route to PCA
<code>np.dot / @</code>	Unit 3	Vectorized regression hypothesis, cost function
<code>.T / transpose</code>	Unit 3	Normal equation, gradient computation
<code>np.linalg.inv</code>	Unit 3	Closed-form (Ridge) regression solution

<code>np.sum</code>	Unit 3	Cost function, R^2 , gradient sums
<code>np.random.randn/rand</code>	Unit 3	Weight init, shuffling data
<code>np.exp</code>	Unit 4	Sigmoid, softmax, Gaussian likelihood
<code>np.log</code>	Unit 4	Log-loss, Naive Bayes log-probabilities
<code>np.where</code>	Unit 4	Thresholding, perceptron/SVM decision rule
<code>np.argmax/argmin</code>	Unit 4	Class choice, decision tree split, nearest label
<code>np.linalg.norm</code>	Unit 4 & 5	Distance metric for KNN and clustering
<code>np.sort/argsort</code>	Unit 4	Rank neighbors by distance for KNN
<code>np.unique/bincount</code>	Unit 4	Class priors, KNN majority vote
<code>np.maximum</code>	Unit 4	ReLU activation, SVM hinge loss
<code>np.mean (per cluster)</code>	Unit 5	K-means centroid update step
<code>np.argmin (per point)</code>	Unit 5	K-means cluster assignment step
<code>np.random.choice</code>	Unit 5	Random initial centroids

Combined Capstone Exercise

Ties every unit together in one pipeline, mirroring exactly what the practicals section of your syllabus asks for.

END-TO-END MINI PROJECT

1. Pick one dataset from `data.gov.in` or the UCI repository with at least 4 numeric features and a target column suited to either regression or classification.
2. Preprocess it: standardize with mean/std, then reduce to 2 dimensions with PCA (`cov + linalg.eig`).
3. Split into train/test sets using array slicing after shuffling with `np.random.permutation`.
4. Train *either* a linear regression model (Normal Equation with `linalg.inv/.T/@`) *or* a logistic regression / KNN classifier (`exp, log, linalg.norm, argsort`).

5. Evaluate it: MSE + R^2 for regression, or Accuracy/Precision/Recall/F1 for classification, all computed with boolean masks and `np.sum/np.mean` — no external metrics library.
6. Finally, ignore the labels and run your from-scratch K-means (`random.choice`, `linalg.norm`, `argmin`, `mean`) on the same standardized features. Do the clusters roughly line up with your original target classes?